

API Testing & Results

Done By : Shahd AL Abdali

Add Customer

The screenshot displays the Postman application interface. On the left, the 'My Collection' sidebar shows a 'GET Get data' request selected. The main workspace shows a 'POST Get data' request to the endpoint `http://localhost:8080/customers/add`. The request body is a JSON object with the following fields: `customerName` (Ahmed), `accountNumber` (12345), `balance` (500), `email` (ahmed@gmail.com), and `phoneNumber` (99999999). The response is a 200 OK status with a response time of 398 ms and a body of 287 B. The response body is displayed in JSON format, showing the same fields as the request body, with an additional `customerId` field set to 1. The interface also includes a 'New Chat' panel on the right and a system tray at the bottom.

My Collection > Get data

POST `http://localhost:8080/customers/add` Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON Schema Beautify

```
1 {
2   "customerName": "Ahmed",
3   "accountNumber": "12345",
4   "balance": 500,
5   "email": "ahmed@gmail.com",
6   "phoneNumber": "99999999"
7 }
```

Body Cookies Headers 5 Test Results 1/1 200 OK • 398 ms • 287 B Save Response

{ JSON Preview Visualize

```
1 {
2   "customerId": 1,
3   "customerName": "Ahmed",
4   "accountNumber": "12345",
5   "balance": 500.0,
6   "email": "ahmed@gmail.com",
7   "phoneNumber": "99999999"
8 }
```

Fix the 400 response parsing

Add a request/body template

Turn failed inputs into tests

Describe what you need. Press @ for context, / for Skills.

Auto

Connect Git Console Terminal 0 0

Hot days ahead 30°C

Search

Globals Vault Tools

3:43 PM 6/10/2026

Get All Customers

The screenshot displays the Postman API client interface. The left sidebar shows a collection named "My Collection" with two items: "GET Get data" (selected) and "POST Post data". The main workspace shows the details of the "GET Get data" request. The URL is "http://localhost:8080/customers/getAll". The request is a GET method with no body. The response is a 200 OK status with a response time of 25 ms and a body size of 548 B. The response body is a JSON array of three customer objects, each containing fields like customerId, customerName, accountNumber, balance, email, and phoneNumber. The bottom status bar shows "Connect Git", "Console", "Terminal", and "0 0 0". The right sidebar shows a "New Chat" button and a chat input area.

Overview Overview GET Get data POST Post data No environment

My Collection > Get data Save Share

GET http://localhost:8080/customers/getAll Send

Docs Params Authorization Headers 6 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

1

Body Cookies Headers 5 Test Results 1/1 200 OK • 25 ms • 548 B Save Response

{ } JSON Preview Visualize

```
1 [
2   {
3     "customerId": 1,
4     "customerName": "Ahmed",
5     "accountNumber": "12345",
6     "balance": 500.0,
7     "email": "ahmed@gmail.com",
8     "phoneNumber": "99999999"
9   },
10  {
11    "customerId": 2,
12    "customerName": "Ali",
13    "accountNumber": "1002",
14    "balance": 1200.0,
15    "email": "ali@gmail.com",
16    "phoneNumber": "88888888"
17  },
18  {
19    "customerId": 3,
20    "customerName": "Ahmed",
21    "accountNumber": "1003",
22    "balance": 2500.0,
23    "email": "ahmed2@gmail.com",
24    "phoneNumber": "77777777"
25  }
26 ]
```

ENVIRONMENTS SPECS FLOWS

Connect Git Console Terminal 0 0 0

Globals Vault Tools

New Chat

Turn Get data tests into assertions

Create a Data file + runner

Add a POST request contract test

Describe what you need. Press @ for context, / for Skills.

Auto

Get Customer By ID

The screenshot displays the Postman application interface. On the left, the 'COLLECTIONS' sidebar shows 'My Collection' with two items: 'GET Get data' (selected) and 'POST Post data'. The main workspace is titled 'My Collection > Get data'. The request method is 'GET' and the URL is 'http://localhost:8080/customers/getById?id=1'. The 'Body' tab is active, showing a 'raw' request body with the number '1'. The response status is '200 OK' with a response time of '48 ms' and a size of '287 B'. The response body is displayed in JSON format:

```
{
  "customerId": 1,
  "customerName": "Ahmed",
  "accountNumber": "12345",
  "balance": 500.0,
  "email": "ahmed@gmail.com",
  "phoneNumber": "999999999"
}
```

At the bottom, the Windows taskbar is visible, showing the system clock as 3:49 PM on 6/10/2026.

Get Customer By Name

The screenshot displays the Postman application interface. On the left, a sidebar shows a collection named 'My Collection' with two items: 'GET Get data' (selected) and 'POST Post data'. The main workspace is divided into several tabs: 'Overview', 'Params', 'Authorization', 'Headers', 'Body', 'Scripts', 'Tests', 'Settings', and 'Cookies'. The 'Body' tab is active, showing a JSON response. The request is a GET request to 'http://localhost:8080/customers/getByName?customerName=Ahmed'. The response status is '200 OK' with a response time of '557 ms' and a size of '421 B'. The JSON response is as follows:

```
1 [
2   {
3     "customerId": 1,
4     "customerName": "Ahmed",
5     "accountNumber": "12345",
6     "balance": 500.0,
7     "email": "ahmed@gmail.com",
8     "phoneNumber": "99999999"
9   },
10  {
11    "customerId": 3,
12    "customerName": "Ahmed",
13    "accountNumber": "1003",
14    "balance": 2500.0,
15    "email": "ahmed2@gmail.com",
16    "phoneNumber": "77777777"
17  }
18 ]
```

On the right side of the interface, there is a 'New Chat' section with a text input area and a 'Send' button. Below the input area, there are several suggestions: 'Turn Get data tests into assertions', 'Create a Data file + runner', and 'Add a POST request contract test'. At the bottom right, there is a 'Describe what you need. Press @ for context, / for Skills.' section with an 'Auto' dropdown and a 'Send' button.

Update Customer

The screenshot displays the Postman API client interface. The top navigation bar includes a search bar, 'Invite', 'Upgrade', and various utility icons. The left sidebar shows a workspace named 'SHAHD's Workspace' with a 'COLLECTIONS' panel containing 'My Collection' and two items: 'GET Get data' and 'POST Post data'. The main workspace is divided into two tabs: 'Overview' and 'PUT Get data'. The 'PUT Get data' tab is active, showing a request to 'http://localhost:8080/customers/update?id=1'. The request body is a JSON object with customer details, and the response is a 200 OK status with a JSON object containing the updated customer information. The right sidebar features a 'New Chat' section with a chat input and a 'Turn Get data tests into assertions' button.

SHAHD's Workspace

Overview

PUT Get data

POST Post data

My Collection > Get data

PUT http://localhost:8080/customers/update?id=1

Docs Params Authorization Headers 8 Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "customerName": "Ahmed Ali",
3   "accountNumber": "12345",
4   "balance": 1000,
5   "email": "ahmedali@gmail.com",
6   "phoneNumber": "77777777"
7 }
```

Body Cookies Headers 5 Test Results 1/1

200 OK • 53 ms • 295 B • Save Response

{ } JSON Preview Visualize

```
1 {
2   "customerId": 1,
3   "customerName": "Ahmed Ali",
4   "accountNumber": "12345",
5   "balance": 1000.0,
6   "email": "ahmedali@gmail.com",
7   "phoneNumber": "77777777"
8 }
```

ENVIRONMENTS

SPECS

FLows

Connect Git Console Terminal 0 0

Globals Vault Tools

Turn Get data tests into assertions

Create a Data file + runner

Add a POST request contract test

Describe what you need. Press @ for context, / for Skills.

Auto

Validation (Negative Balance)

The screenshot displays the Postman application interface. On the left, the 'COLLECTIONS' sidebar shows 'My Collection' with two items: 'GET Get data' and 'POST Post data'. The main workspace is titled 'My Collection > Get data'. The request is a POST to 'http://localhost:8080/customers/add'. The 'Body' tab is selected, showing a raw JSON payload:

```
1 {
2   "customerName": "Ahmed",
3   "accountNumber": "1005",
4   "balance": -100,
5   "email": "ahmed@gmail.com",
6   "phoneNumber": "99999999"
7 }
```

The response status is '400 Bad Request' with a duration of 23 ms and a size of 243 B. The 'Body' tab shows the JSON response:

```
1 {
2   "timestamp": "2026-06-10T12:17:56.387Z",
3   "status": 400,
4   "error": "Bad Request",
5   "path": "/customers/add"
6 }
```

At the bottom right, there are three suggestions: 'Fix localhost connection for request', 'Add error-handling tests for 4xx/5xx', and 'Turn Post data into contract tests'. A chat interface is also visible on the far right with the text 'Describe what you need. Press @ for context, / for Skills.'

Validation (Name Empty)

The screenshot displays a REST client interface with a workspace named "SHAHD's Workspace". The left sidebar shows a collection of requests: "GET Get data" and "POST Post data". The main panel is set to "POST" with the URL "http://localhost:8080/customers/add". The request body is a JSON object:

```
1 {
2   "customerName": "",
3   "accountNumber": "1005",
4   "balance": 100,
5   "email": "ahmed@gmail.com",
6   "phoneNumber": "99999999"
7 }
```

The response status is "400 Bad Request" with a 12 ms duration and 243 B of data. The response body is a JSON object:

```
1 {
2   "timestamp": "2026-06-10T12:19:28.890Z",
3   "status": 400,
4   "error": "Bad Request",
5   "path": "/customers/add"
6 }
```

The bottom status bar shows "Connect Git", "Console", "Terminal", and "0 0 0". The system tray at the bottom indicates a temperature of 38°, a search bar, and the time 4:19 PM on 6/10/2026.

MySQL Database Table

The screenshot displays the MySQL Workbench interface for a local instance of MySQL 8.0. The 'Navigator' pane on the left shows the database structure, including the 'customers' table. The 'Query' pane in the center contains the SQL query: `SELECT * FROM banking.customers;`. The 'Result Grid' pane shows the query results, which include three rows of customer data. The 'Table: customers' pane on the bottom left lists the columns and their data types. The 'Output' pane at the bottom right shows the execution log, indicating that the query was executed successfully and returned 3 rows.

Table: customers

Columns:

- customer_id: int PK
- account_number: varchar
- balance: double
- customer_name: varchar
- email: varchar
- phone_number: varchar

Query Results:

customer_id	account_number	balance	customer_name	email	phone_number
1	12345	500	Ahmed	ahmed@gmail.com	99999999
2	1002	1200	Ali	ali@gmail.com	88888888
3	1003	2500	Ahmed	ahmed2@gmail.com	77777777

Execution Log:

#	Time	Action	Message	Duration / Fetch
1	15:44:55	SELECT * FROM banking.customers LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
2	15:48:39	SELECT * FROM banking.customers LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec